```
        x = self.conv3(x)
        x = nn.functional.relu(x)
        x = self.conv4(x)
        x = nn.functional.relu(x)
        x = self.conv5(x)
        x = nn.functional.relu(x)
        x = x.flatten(start_dim=1)
        x = self.lin1(x)
        x = nn.functional.relu(x)
        x = self.out(x)
        x = nn.functional.log_softmax(x,dim=1)              4
        return x
```

- ***1*** The architecture consists of 5 convolutional layers total.
- ***2*** After the first 4 convolutional layers, we MaxPool to reduce the dimensionality.
- ***3*** The last layer is a linear layer after we flatten the output from the CNN.
- ***4*** Lastly, we apply the log_softmax function to classify the digits probabilistically.

Instantiate this CNN and swap it in for the relational module in the previous training loop to see how it compares. We get a test accuracy of only 87.80% with this CNN, demonstrating that our relational module is outperforming a CNN architecture, controlling for the number of parameters. Moreover, if you crank up the transformation level (e.g., add more noise, rotate even more), the relational module will maintain a higher accuracy than the CNN. As we noted earlier, our particular implementation of the relational module is not practically invariant to rotations and deformations because, in part, we've added the absolute coordinate positions; it's not all relational, but it has the ability to compute long-distance relations between features in the image, as opposed to a CNN that can just compute local features.

We wanted to introduce relational modules not merely because they might get better accuracy on some data set, but because they are more interpretable than traditional neural network models. We can inspect the learned relationships in the attention weight matrix to see which parts of the input the relational module is using to classify images or predict Q values as shown in figure 10.17.